Associer des données avec Pandas : les jointures

Lino Galiana
Version du 20 octobre 2026

Le chapitre d'introduction à `Pandas` a permis de présenter le principe de données organisées sous une forme de *DataFrame* et la praticité de l'écosystème `Pandas` pour effectuer des opérations simples sur un jeu de données. Ce chapitre consolide ces principes en présentant l'une des opérations les plus fréquentes en *data science* : l'association de données par le biais d'une jointure.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **2**
1.1 Environnement 2
1.2 Données utilisées 2
2 Récupération des jeux de données **4**
2.1 Données d'émission de l'Ademe 4
2.2 Données *Filosofi* 4
3 Le principe du schéma en étoile **6**
3.1 Introduction au schéma en étoile 6
3.2 Les clés de jointures 7
4 Mise en oeuvre avec `Pandas` **8**
4.1 *Left join* 8
4.2 *Right join* 10
4.3 *Inner join* 12
4.4 *Full join* 13
4.5 En résumé 14
5 Exemples d'identifiants dans les données françaises **15**
5.1 Le Code officiel géographique (COG): l'identifiant des données géographiques 15
5.2 Sirene: l'identifiant dans les données d'entreprises 17
6 Exercices d'application **21**
6.1 Pourquoi a-t-on besoin d'un code commune quand on a déjà son nom ? 21
6.2 Calculer une empreinte carbone grâce à l'association entre des sources 21
Informations additionnelles 22
Bibliographie **24**

Compétences à l'issue de ce chapitre

- Récupérer un jeu de données officiel de l'Insee ;
- Comprendre la logique d'une jointure et savoir choisir le type de pivot adapté ;
- Associer plusieurs `DataFrames` à partir de caractéristiques communes ;
- Connaître les identifiants permettant de relier entre elles de multiples sources françaises.

Les exercices de mise en pratique dans ce tutoriel sont assez tardifs, une fois les principes des jointures et des identifiants passés en revue : voir Tip 1 et Tip 2.

1 Introduction

Le chapitre d'introduction à `Pandas` a permis de présenter le principe de données organisées sous une forme de *DataFrame* et la praticité de l'écosystème `Pandas` pour effectuer des opérations simples sur un jeu de données.

L'un des apports des outils modernes de *data science*, notamment `Pandas`, est la simplicité par laquelle ils permettent de restructurer des sources pour travailler sur plusieurs données sur un projet.

Il est rare de travailler exclusivement sur une source brute. Un jeu de données prend généralement de la valeur lorsqu'il est comparé à d'autres sources. Pour des chercheurs, cela permettra de contextualiser l'information présente dans une source en la comparant ou en l'associant à d'autres sources. Pour des *data scientists* dans le secteur privé, il s'agira souvent d'associer des informations sur une même personne dans plusieurs bases clientes ou comparer les clients entre eux.

Effectuer ce travail de manière simple, fiable et efficace est indispensable pour les *data scientists* tant cette tâche est courante. Heureusement `Pandas` permet de faire cela très bien avec des données structurées. Nous verrons dans les prochains chapitres, mais aussi dans l'ensemble de la partie sur le traitement des données textuelles, comment faire avec des données moins structurées.

Grâce à ce travail d'enrichissement, nous serons en mesure, dans le prochain chapitre, d'approfondir notre compréhension d'un phénomène réel par le biais de statistiques descriptives fines. Cela est une étape indispensable avant de basculer vers la statistique inférentielle, l'approche qui consiste à formaliser et généraliser des liens de corrélation ou de causalité entre des caractéristiques observées et un phénomène.

1.1 Environnement

Le chapitre précédent utilisait quasi exclusivement la librairie `Pandas` car nous avions une source prête à l'emploi. Dans ce chapitre, nous aurons besoin d'aller chercher des données un peu moins accessibles : cela nécessitera un peu plus de travail.

Les principes généraux de la récupération de données seront présentés dans le chapitre consacré aux API. Nous donnerons directement le code de récupération des données dans ce chapitre pour pouvoir se consacrer au coeur de notre problème: associer ces différentes données entre-elles.

Les premiers *packages* indispensables pour démarrer ce chapitre sont les suivants:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import requests
```

Pour obtenir des résultats reproductibles, on peut fixer la racine du générateur pseudo-aléatoire.

```
import numpy as np
np.random.seed(123)
```

1.2 Données utilisées

Ce tutoriel continue l'exploration du jeu de données du chapitre précédent:

- Les émissions de gaz à effet de serre estimées au niveau communal par l'ADEME. Le jeu de données est disponible sur data.gouv et requêtable directement dans `Python` avec cet url ;

Les problématiques d'enrichissement de données (association d'une source à une autre à partir de caractéristiques communes) seront présentées à partir de deux sources produites par l'Insee:

- Le code officiel géographique, un référentiel produit par l'Insee utilisé pour identifier les communes à partir d'un code unique, contrairement au code postal ;
- Les données *Filosofi*, une source sur les revenus des Français à une échelle spatiale fine construite par l'Insee à partir des déclarations fiscales et d'informations sur les prestations sociales. En l'occurrence, nous allons utiliser, au niveau communal, des données de revenu médian et la population totale[1]. Cela nous aidera à calculer de nouveaux indicateurs d'émissions et à les interpréter.

Pour importer ces données, nous allons directement interroger l'API `Melodi`, le nouveau point d'entrée mis à disposition par l'Insee pour diffuser ses principales sources de données, dont fait partie *Filosofi*.

> **i** Récupérer des données open data produites par l'Insee
>
> Une précédente version de ce tutoriel s'appuyait sur le *package* communautaire `pynsee` qui simplifie l'accès aux données officielles de l'Insee.
>
> Cependant, les sources qui nous sont ici utiles ont été intégrées dans une nouvelle API de l'Insee nommée `Melodi`. Comme c'est un produit officiel de l'offre de diffusion de l'Insee, on va privilégier celle-ci. L'inconvénient est que nous n'avons pas, comme avec `pynsee` , un *dataframe* prêt à l'emploi. Le code de préparation des données sera donc un peu plus complexe.
>
> L'API renvoie des données structurées au format `JSON`: chaque observation associe des dimensions (par exemple `GEO` pour la zone géographique ou `TIME_PERIOD` pour la période) à une valeur (`OBS_VALUE_NIVEAU`). Plus d'éléments sur l'exploitation de JSON via Python seront proposés dans le chapitre consacré aux API.
>
> Chaque source dispose d'un identifiant unique (par exemple `DS_FILOSOFI_CC` pour les données Filosofi communales, `DS_POPULATIONS_HISTORIQUES` pour les populations légales). Le point d'entrée principal est de la forme `https://api.insee.fr/melodi/data/{identifiant}`, que l'on peut filtrer par dimension (zone géographique, période, indicateur...). La structure de chaque fichier dépend de la source demandée. Il y a donc généralement un travail propre à chaque source à mettre en oeuvre.
>
> Explorer la structure d'une source Pour connaître la structure d'une source (dimensions, indicateurs disponibles...), on peut consulter ses métadonnées, également disponibles au format `JSON` :
>
> ```
> import requests
> requests.get("https://api.insee.fr/melodi/catalog/DS_FILOSOFI_CC").json()
> ```
>
> La liste des sources disponibles sur l'API Melodi est quant à elle consultable sur le portail des API de l'Insee.

[1] Pour les données démographiques, on utilise directement les populations légales diffusées par l'Insee, issues du recensement, plutôt qu'un indicateur de population fiscale. Un exercice d'ouverture est également proposé pour construire des agrégats de population à partir des jeux de données individuels anonymisés du recensement (les fichiers détails).

2 Récupération des jeux de données

2.1 Données d'émission de l'Ademe

Comme expliqué au chapitre précédent, ces données peuvent être importées très simplement avec `Pandas`

```
import pandas as pd

url   =   "https://data.ademe.fr/data-fair/api/v1/datasets/igt-pouvoir-de-rechauffement-global/
convert"
emissions = pd.read_csv(url)
emissions.head(2)
```

	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire
0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172
1	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207

Nous allons d'ores et déjà conserver le nom des secteurs émetteurs présents dans la base de données pour simplifier des utilisations ultérieures:

```
secteurs = emissions.select_dtypes(include='number').columns
secteurs
```

	columns
0	Agriculture
1	Autres transports
2	Autres transports international
3	CO2 biomasse hors-total
4	Déchets
5	Energie
6	Industrie hors-énergie
7	Résidentiel
8	Routier
9	Tertiaire

Les exploitations ultérieures de ces données utiliseront la dimension départementale dont nous avons montré la construction au chapitre précédent:

```
emissions['dep'] = emissions["INSEE commune"].str[:2]
```

2.2 Données *Filosofi*

On va utiliser les données Filosofi (données de revenus) au niveau communal, dans leur millésime le plus récent (2023 au moment de la rédaction de ce cours). Ce n'est pas la même année que les données d'émission de l'Ademe (régulièrement mises à jour, elles portent sur 2025), ce n'est donc pas parfaitement rigoureux, mais cela permettra tout de même d'illustrer les principales fonctionnalités de `Pandas`.

Au niveau communal, la source Melodi `DS_FILOSOFI_CC` ne diffuse que le niveau de vie médian (`MED_SL`) et le taux de pauvreté (`PR_MD60`) ; les autres indicateurs (déciles, indice de Gini, décomposition du revenu...) ne sont disponibles qu'aux niveaux département, région et France entière. On y ajoute la

population municipale, disponible sur la source Melodi `DS_POPULATIONS_HISTORIQUES`, ainsi que le nom des communes, issu du code officiel géographique.

Le code pour récupérer ces données est le suivant :

```
import requests
import pandas as pd

def get_melodi(dataset, dimension, code, value_name, time_period, geo_level="COM"):

  url = f"https://api.insee.fr/melodi/data/{dataset}"

  params = {
    "GEO": geo_level,
    dimension: code,
    "TIME_PERIOD": time_period,
    "maxResult": 40000,
  }

  response = requests.get(url, params=params, timeout=60)
  response.raise_for_status()
  observations = response.json()["observations"]

  return pd.DataFrame(
    {
      "CODGEO": obs["dimensions"]["GEO"].split("-")[-1],
      value_name: obs["measures"]["OBS_VALUE_NIVEAU"].get("value"),
    }
    for obs in observations
  )

niveau_vie = get_melodi("DS_FILOSOFI_CC", "FILOSOFI_MEASURE", "MED_SL", "NIVVIE_MEDIAN", 2023)
taux_pauvrete = get_melodi("DS_FILOSOFI_CC", "FILOSOFI_MEASURE", "PR_MD60", "TAUX_PAUVRETE",
2023)
population = get_melodi("DS_POPULATIONS_HISTORIQUES", "POPREF_MEASURE", "PMUN", "POPULATION",
2023)

url_cog_2023 = "https://www.insee.fr/fr/statistiques/fichier/6800675/v_commune_2023.csv"
url_backup = "https://minio.lab.sspcloud.fr/lgaliana/data/python-ENSAE/cog_2023.csv"

try:
  cog_2023 = pd.read_csv(url_cog_2023)
except requests.exceptions.Timeout:
  cog_2023 = pd.read_csv(url_backup)

noms_communes = (
  cog_2023
  .loc[cog_2023["TYPECOM"] == "COM", ["COM", "LIBELLE"]]
  .rename(columns={"COM": "CODGEO", "LIBELLE": "LIBGEO"})
)

filosofi = (
  niveau_vie
  .merge(taux_pauvrete, on="CODGEO", how="outer")
  .merge(population, on="CODGEO", how="outer")
  .merge(noms_communes, on="CODGEO", how="left")
  [["CODGEO", "LIBGEO", "POPULATION", "NIVVIE_MEDIAN", "TAUX_PAUVRETE"]]
)
```

```
filosofi["POPULATION"] = filosofi["POPULATION"].astype(int)
filosofi["dep"] = filosofi["CODGEO"].str[:2]
```

Le *DataFrame* en question a l'aspect suivant :

```
filosofi.sample(3)
```

	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE	dep
31453	80574	Mouflers	96	NaN	NaN	80
21028	57332	Hombourg-Haut	5976	21870.0	24.0	57
30870	79289	Saint-Pierre-des-Échaubrognes	1384	25800.0	NaN	79

`Pandas` a géré automatiquement les types de variables. Il le fait relativement bien, mais s'il y a un doute, il est toujours possible d'utiliser la méthode `dtypes` pour vérifier.

Un simple coup d'oeil sur les données donne une idée assez précise de la manière dont les données sont organisées. On remarque que certaines variables de `filosofi` semblent avoir beaucoup de valeurs manquantes (secret statistique) alors que d'autres semblent complètes. Si on désire exploiter `filosofi`, il faut faire attention à la variable choisie.

Notre objectif immédiat va donc être de relier l'information contenue entre nos deux jeux de données. Sans cela, nous risquons d'être frustré : nous allons vouloir en savoir plus sur les émissions de gaz carbonique mais seront très limités dans les possibilités d'analyse sans ajout d'une information annexe issue de `filosofi`. C'est cet enrichissement, par le biais d'une jointure, que nous allons mettre en oeuvre dès maintenant.

3 Le principe du schéma en étoile

3.1 Introduction au schéma en étoile

Un jeu de données se suffit rarement à lui-même. Pour donner du sens et de la valeur à une statistique, il faut généralement l'associer à de la connaissance annexe, sous peine qu'elle reste désincarnée. Ce type de croisement, qu'on appelle un enrichissement de données, est un besoin quotidien des *data scientists* et l'une des opérations les plus fréquentes en *data science*.

Il peut se faire à un niveau identique à celui de la source d'origine (par exemple relier, dans une base d'entreprise, les informations clients à celles éparpillées entre un fichier ventes et un fichier produits) ou à des niveaux conceptuels différents, en général plus agrégés, pour contextualiser une donnée plus fine (par exemple associer des temps de transport individuels à ceux d'une même classe d'âge ou de personnes résidant dans la même commune).

Nous allons ici nous focaliser sur le cas le plus favorable qui est la situation où une information permet d'apparier de manière exacte deux bases de données[2].

[2] Autrement, on rentre dans le monde des appariements flous ou des appariements probabilistes.

Les appariements flous sont des situations où on ne dispose plus d'un identifiant exact pour associer deux bases mais d'une information partiellement bruitée entre deux sources pour faire cette mise en relation. Par exemple, dans une base de données produit on aura `Coca Cola 33CL` et dans une autre `Coca Cola canette` mais sous ces deux noms sont cachés le même produit. On utilisera par exemple des techniques de distance textuelle pour relier les observations les unes aux autres. C'est, typiquement, ce que fait un moteur de recherche.

Les appariements probabilistes sont un autre type d'approche. Dans ceux-ci, on associe des observations dans deux bases non pas sur la base d'un identifiant mais sur la distance entre un ensemble de caractéristiques dans les deux bases. Cette technique est très utilisée dans les statistiques médicales ou dans l'évaluation de politiques publiques sur la base du *propensity score matching*.

Cette pratique découle du fait que de nombreux systèmes d'information prennent la forme d'un schéma en étoile (Figure 1):

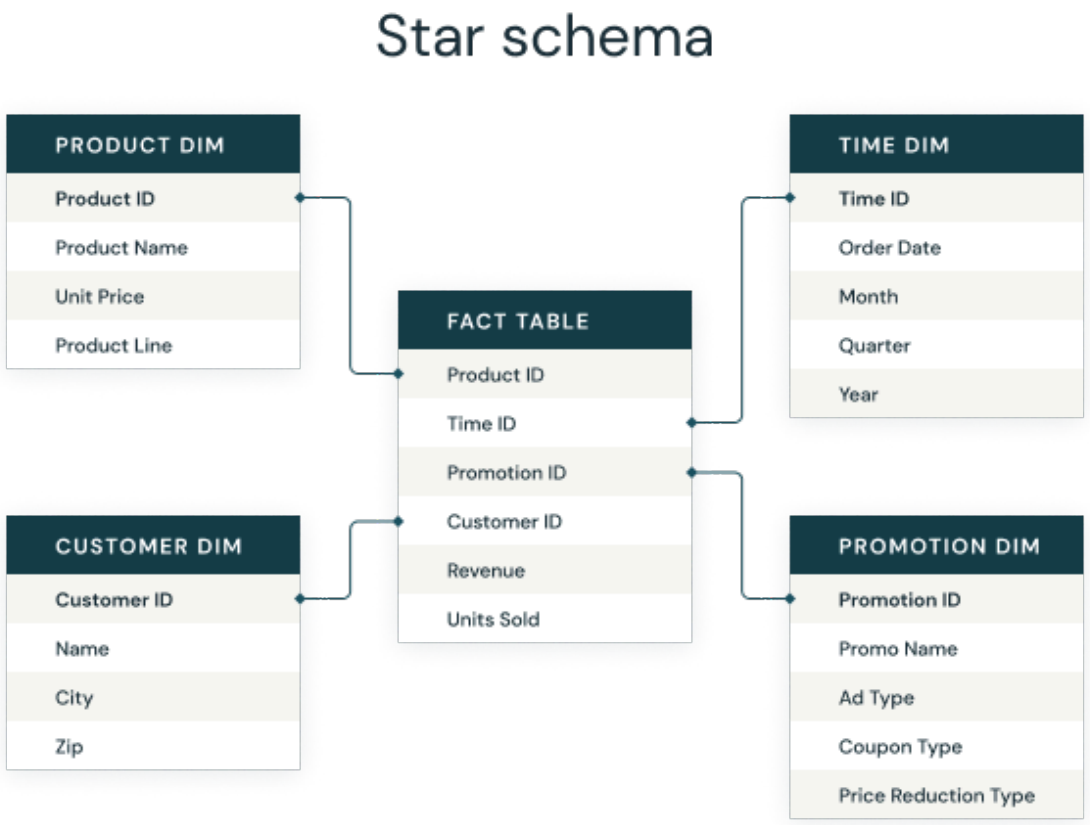


Figure 1. – Illustration du schéma en étoile (Source: Databricks)

Cette structuration de l'information est très liée au modèle des tables relationnelles des années 1980. Aujourd'hui, il existe des modèles de données plus flexibles où l'information est empilée dans un *data lake* sans structure *a priori*. Néanmoins ce modèle du schéma en étoile reste pertinent parce qu'il permet d'organiser l'information autour d'unités statistiques d'intérêt (le client, l'entreprise, la transaction, etc.) tout en permettant un cloisonnement des données.

Puisque la logique du schéma en étoile vient historiquement des bases relationnelles, il est naturel qu'il s'agisse d'une approche intrinsèquement liée à la philosophie du SQL, jusque dans le vocabulaire. On parle souvent de jointure de données, un héritage du terme `JOIN` de SQL, et la manière de décrire les jointures (*left join*, *right join*...) est directement issue des instructions SQL associées.

3.2 Les clés de jointures

On parle de **clé(s) de jointure** pour nommer la ou les variable(s) nécessaire(s) à la fusion de données. Ce sont les variables communes aux deux jeux de données. Il n'est pas nécessaire qu'elles aient le même nom en revanche elles doivent partager des valeurs communes autrement l'intersection entre ces deux bases est l'ensemble vide.

On peut jouer sur deux dimensions dans la jointure (ceci sera plus clair ensuite avec les exemples graphiques).

- Il existe principalement trois types de fusions: ***left join*** et ***right join*** ou un combo des deux selon le type de pivot qu'on désire mettre en oeuvre.
- Ensuite, il existe deux manières de fusionner les valeurs une fois qu'on a choisi un pivot: ***inner join*** ou ***outer join***. Dans le premier cas, on ne conserve que les observations où les clés de

jointures sont présentes dans les deux bases, dans le second on conserve toutes les observations de la clé de jointure des variables pivot quitte à avoir des valeurs manquantes si la deuxième base de données n'a pas de telles observations.

Dans les exemples ci-dessous, nous allons utiliser les codes communes et les départements comme variables de jointure. En soi, l'usage du département n'est pas nécessaire puisqu'il se déduit directement du code commune mais cela permet d'illustrer le principe des jointures sur plusieurs variables. A noter que le nom de la commune est volontairement mis de côté pour effectuer des jointures alors que c'est une information commune aux deux bases. Cependant, comme il s'agit d'un champ textuel, dont le formattage peut suivre une norme différente dans les deux bases, ce n'est pas une information fiable pour faire une jointure exacte.

Pour illustrer le principe du pivot à gauche ou à droite, on va créer deux variables identificatrices de la ligne de nos jeux de données de gauche et de droite. Cela nous permettra de trouver facilement les lignes présentes dans un jeu de données mais pas dans l'autre.

```
emissions = emissions.reset_index(names = ['id_left'])
filosofi = filosofi.reset_index(names = ['id_right'])
```

4 Mise en oeuvre avec `Pandas`

En `Pandas`, la méthode la plus pratique pour associer des jeux de données à partir de caractéristiques communes est `merge`. Ses principaux arguments permettent de contrôler le comportement de jointure. Nous allons les explorer de manière visuelle.

En l'occurrence, pour notre problématique de construction de statistiques sur les émissions de gaz carbonique, la base de gauche sera le *DataFrame* `emission` et la base de droite le *DataFrame* `filosofi`:

```
emissions.head(2)
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep
0	0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172	01
1	1	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01

```
filosofi.head(2)
```

	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE	dep
0	0	01001	L'Abergement-Clémenciat	860	28270.0	NaN	01
1	1	01002	L'Abergement-de-Varey	270	28140.0	NaN	01

4.1 *Left join*

Commençons avec la jointure à gauche. Comme son nom l'indique, on va prendre la variable de gauche en pivot:

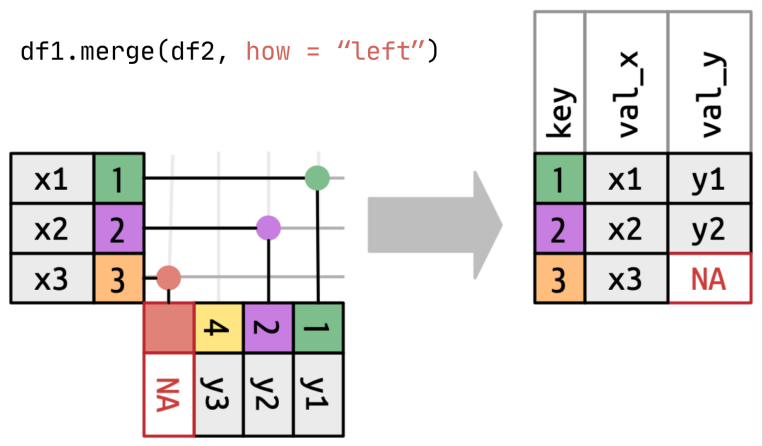


Figure 2. – Illustration du *left join*. Source originale: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1]

```
left_merged = emissions.merge(
  filosofi,
  left_on = ["INSEE commune", "dep"],
  right_on = ["CODGEO", "dep"],
  how = "left"
)
left_merged.head(3)
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE
0	0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172	01	0.0	01001	L'Abergement-Clémenciat	860.0	28270.0	NaN
1	1	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01	1.0	01002	L'Abergement-de-Varey	270.0	28140.0	NaN
2	2	01004	AMBERIEU-EN-BUGEY	499.043526	212.577908	NaN	10313.446515	5314.314445	998.332482	2930.354461	16616.822534	15642.420313	10732.376934	01	2.0	01004	Ambérieu-en-Bugey	15934.0	24210.0	18.0

Il est recommandé de toujours expliciter les clés de jointures par le biais des arguments `left_on`, `right_on` ou `on` si les noms de variables sont communs dans les deux bases. Si on a des noms de variables communes entre les bases mais qu'elles ne sont pas définies comme clés de jointures, celles-ci ne seront pas utilisées pour joindre mais seront conservées avec un suffixe qui par défaut est `_x` et `_y` (paramétrable par le biais de l'argument `suffixes`).

La syntaxe `Pandas` étant directement inspirée de SQL, on a une traduction assez transparente de l'instruction ci-dessus en SQL:

```
SELECT *
FROM emissions
LEFT JOIN filosofi
  ON emissions.`INSEE commune` = filosofi.CODGEO
  AND emissions.dep = filosofi.dep;
```

En faisant une jointure à gauche, on doit en principe avoir autant de lignes que la base de données à gauche:

```
left_merged.shape[0] == emissions.shape[0]
```

```
True
```

Autrement, cela est signe qu'il y a une clé dupliquée à droite. Grâce à notre variable `id_right`, on peut savoir les codes communes à droite qui n'existent pas à gauche:

```
left_merged.loc[left_merged['id_right'].isna()].tail(3)
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE
35552	35552	93059	PIERREFITTE-SUR-SEINE	3.384360	293.997505	NaN	8462.785892	63344.922645	915.922961	2688.461993	22146.952752	24167.708776	14417.152611	93	NaN	NaN	NaN	NaN	NaN	NaN
35687	35687	95259	GADANCOURT	312.298700	NaN	NaN	142.113291	11.372909	NaN	NaN	32.440647	2060.981036	41.153991	95	NaN	NaN	NaN	NaN	NaN	NaN
35693	35693	95282	GOUZANGREZ	826.234401	NaN	NaN	53.146172	23.274790	2.354558	6.911213	55.638159	36.040818	84.222120	95	NaN	NaN	NaN	NaN	NaN	NaN

Cela vient du fait que les deux jeux de données ne reposent pas sur la même version du code officiel géographique. Les données Filosofi, diffusées par l'API Melodi, reflètent le découpage communal le plus récent (2025), alors que les données d'émissions de l'Ademe, bien que mises à jour en 2025, n'ont pas encore répercuté certaines fusions de communes récentes. Par exemple, la commune de Courcouronnes est toujours présente telle quelle dans les données d'émissions (base de gauche):

```
left_merged.loc[
  left_merged['id_right'].isna()
  & left_merged['Commune'].str.contains("COURCOURONNES", na=False)
]
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE
35348	35348	91182	COURCOURONNES	24.548795	103.360309	NaN	9623.065698	111.241872	1276.170296	3745.877636	9978.002088	57834.224552	10532.120761	91	NaN	NaN	NaN	NaN	NaN	NaN

Cette commune a pourtant fusionné avec Evry pour former la commune nouvelle d'Évry-Courcouronnes, seule forme sous laquelle elle apparaît désormais dans les données Filosofi (base de droite):

```
filosofi.loc[
  filosofi['LIBGEO']
  .str.lower()
  .str.contains("courcouronnes", na=False)
]
```

	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE	dep
34315	34315	91228	Évry-Courcouronnes	66919	21020.0	28.6	91

Dans un exercice de construction de statistiques publiques, on ne pourrait donc se permettre cette disjonction des années.

4.2 *Right join*

Le principe est le même mais cette fois c'est la base de droite qui est prise sous forme de pivot:

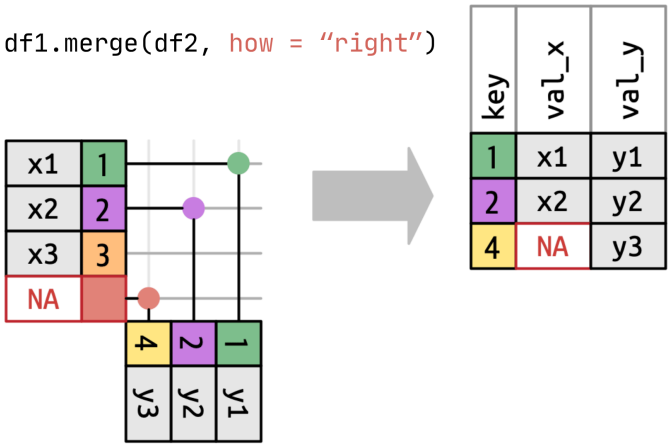


Figure 3. – Illustration du *right join*. Source originale: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1]

```
right_merged = emissions.merge(
  filosofi,
  left_on = ["INSEE commune", "dep"],
  right_on = ["CODGEO", "dep"],
  how = "right"
)
right_merged.head(3)
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE
0	0.0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172	01	0	01001	L'Abergement-Clémenciat	860	28270.0	NaN
1	1.0	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01	1	01002	L'Abergement-de-Varey	270	28140.0	NaN
2	2.0	01004	AMBERIEU-EN-BUGEY	499.043526	212.577908	NaN	10313.446515	5314.314445	998.332482	2930.354461	16616.822534	15642.420313	10732.376934	01	2	01004	Ambérieu-en-Bugey	15934	24210.0	18.0

L'instruction équivalente en SQL serait

```
SELECT *
FROM filosofi
RIGHT JOIN emissions
  ON filosofi.CODGEO = emissions.`INSEE commune`
  AND filosofi.dep = emissions.dep;
```

On peut, comme précédemment, vérifier la cohérence des dimensions:

```
right_merged.shape[0] == filosofi.shape[0]
```

```
True
```

Pour vérifier le nombre de lignes des données Filosofi que nous n'avons pas dans notre jeu d'émissions de gaz carbonique, on peut faire

```
right_merged['id_left'].isna().sum()
```

```
np.int64(120)
```

C'est un nombre faible. Quelles sont ces observations ?

```
right_merged.loc[
  right_merged['id_left'].isna(),
  filosofi.columns.tolist() + emissions.columns.tolist()
]
```

	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE	dep	id_left	INSEE commune	Commune	...	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep
4201	4201	12218	NaN	1540	24390.0	NaN	12	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	12
4338	4338	13055	Marseille	886040	22860.0	28.1	13	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	13
17537	17537	49126	NaN	17162	25770.0	7.0	49	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	49
27039	27039	69123	Lyon	519127	28290.0	18.0	69	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	69
29194	29194	75056	Paris	2103778	33650.0	16.8	75	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	75
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
34853	34853	97420	Sainte-Suzanne	25551	19560.0	34.9	97	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	97
34854	34854	97421	Salazie	7268	15180.0	53.0	97	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	97
34855	34855	97422	Le Tampon	82579	18470.0	38.1	97	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	97
34856	34856	97423	Les Trois-Bassins	7221	18980.0	35.0	97	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	97
34857	34857	97424	Cilaos	5040	15660.0	49.0	97	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	97

Il est suprenant de voir que Paris, Lyon et Marseille sont présents dans la base des statistiques communales mais pas dans celles des émissions. Pour comprendre pourquoi, recherchons dans nos données d'émissions les observations liées à Marseille:

```
emissions.loc[
  emissions["Commune"]
  .str.upper()
  .str.contains('MARSEILLE-')
]
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep
4462	4462	13201	MARSEILLE-1ER-ARRONDISSEMENT	303.836964	850.796099	NaN	18749.902852	2572.980666	2366.477690	6946.201368	29755.542456	35324.129125	18535.087435	13
4463	4463	13202	MARSEILLE-2E-ARRONDISSEMENT	303.836964	83.384844	NaN	14248.781083	2451.230873	2366.477690	6946.201368	18629.702329	35324.129125	11530.097205	13
4464	4464	13203	MARSEILLE-3E-ARRONDISSEMENT	303.836964	18.238987	NaN	20705.839485	2627.535940	2366.477690	6946.201368	32283.391803	35324.129125	21695.331089	13
4465	4465	13204	MARSEILLE-4E-ARRONDISSEMENT	303.836964	84.063314	NaN	21525.141519	2649.386137	2366.477690	107849.501368	34633.530963	35324.129125	22961.055568	13
4466	4466	13205	MARSEILLE-5E-ARRONDISSEMENT	303.836964	NaN	NaN	21220.562849	2641.273890	2366.477690	6946.201368	46460.016400	35324.129125	22498.466001	13
4467	4467	13206	MARSEILLE-6E-ARRONDISSEMENT	303.836964	NaN	NaN	19817.462943	2602.050928	2366.477690	6946.201368	32135.350914	35324.129125	20219.051308	13
4468	4468	13207	MARSEILLE-7E-ARRONDISSEMENT	303.836964	NaN	NaN	17667.589643	2544.306929	2366.477690	6946.201368	26082.234669	35324.129125	17252.687971	13
4469	4469	13208	MARSEILLE-8E-ARRONDISSEMENT	303.836964	NaN	NaN	31033.598562	2905.706707	2366.477690	6946.201368	46457.720220	35324.129125	38019.611886	13
4470	4470	13209	MARSEILLE-9E-ARRONDISSEMENT	303.836964	206.323957	15168.902916	63507.303272	2866.252439	2366.477690	6946.201368	53791.270123	35324.129125	41201.045974	13
4471	4471	13210	MARSEILLE-10E-ARRONDISSEMENT	303.836964	NaN	NaN	23774.329289	2709.963639	2366.477690	6946.201368	39770.211148	35324.129125	27865.799133	13
4472	4472	13211	MARSEILLE-11E-ARRONDISSEMENT	303.836964	149.788396	NaN	24246.711719	2720.116339	2366.477690	11159.128368	41737.276404	35324.129125	32805.823144	13
4473	4473	13212	MARSEILLE-12E-ARRONDISSEMENT	303.836964	34.728224	NaN	25332.342243	2752.276194	2366.477690	6946.201368	44144.355145	35324.129125	28921.206190	13
4474	4474	13213	MARSEILLE-13E-ARRONDISSEMENT	303.836964	NaN	NaN	34480.339077	3001.079306	2366.477690	6946.201368	64717.674768	35324.129125	43333.716539	13
4475	4475	13214	MARSEILLE-14E-ARRONDISSEMENT	303.836964	250.117946	NaN	25588.294079	2758.463641	2366.477690	6946.201368	43612.783924	35324.129125	32142.611435	13
4476	4476	13215	MARSEILLE-15E-ARRONDISSEMENT	303.836964	320.003252	NaN	31565.381887	2919.403353	2366.477690	6946.201368	57187.164876	35324.129125	44748.956297	13
4477	4477	13216	MARSEILLE-16E-ARRONDISSEMENT	303.836964	518.266268	12.311853	12119.260477	2392.479039	2366.477690	20211.201368	12795.918860	35324.129125	10336.618838	13
23522	23522	60387	MARSEILLE-EN-BEAUVAISIS	330.191658	90.910961	NaN	1430.605647	191.355801	127.146118	373.205521	573.997868	2780.093648	692.439819	60

Cela vient du fait que le jeu de données des émissions de l'Ademe propose de l'information sur les arrondissements dans les trois plus grandes villes là où le jeu de données de l'Insee ne fait pas cette décomposition.

4.3 *Inner join*

Il s'agit du jeu de données où les clés sont retrouvées à l'intersection des deux tables.

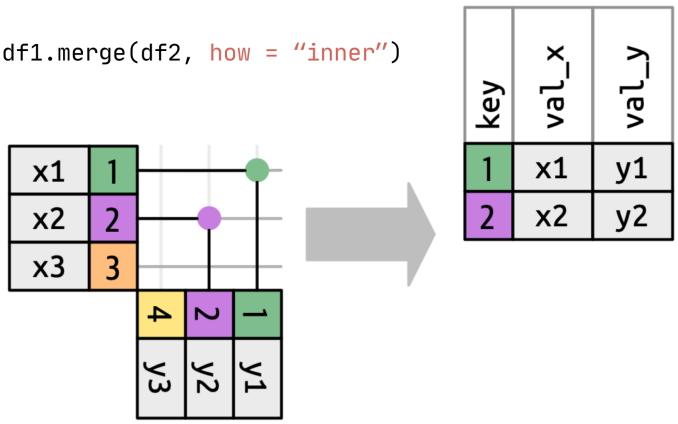


Figure 4. – Illustration du *inner join*. Source originale: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1]

```
inner_merged = emissions.merge(
  filosofi,
  left_on = ["INSEE commune", "dep"],
  right_on = ["CODGEO", "dep"],
  how = "inner"
)
inner_merged.head(3)
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE
0	0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172	01	0	01001	L'Abergement-Clémenciat	860	28270.0	NaN
1	1	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01	1	01002	L'Abergement-de-Varey	270	28140.0	NaN
2	2	01004	AMBERIEU-EN-BUGEY	499.043526	212.577908	NaN	10313.446515	5314.314445	998.332482	2930.354461	16616.822534	15642.420313	10732.376934	01	2	01004	Ambérieu-en-Bugey	15934	24210.0	18.0

En SQL, cela donne

```
SELECT *
FROM emissions
INNER JOIN filosofi
  ON emissions.`INSEE commune` = filosofi.CODGEO
  AND emissions.dep = filosofi.dep;
```

Le nombre de lignes dans notre jeu de données peut être comparé au jeu de droite et de gauche:

```
inner_merged.shape[0] == (
  left_merged.shape[0] - left_merged['id_right'].isna().sum()
)
```

```
np.True_
```

```
inner_merged.shape[0] == (
  right_merged.shape[0] - right_merged['id_left'].isna().sum()
)
```

```
np.True_
```

4.4 *Full join*

Le *full join* est un pivot à gauche puis à droite pour les informations qui n'ont pas été trouvées

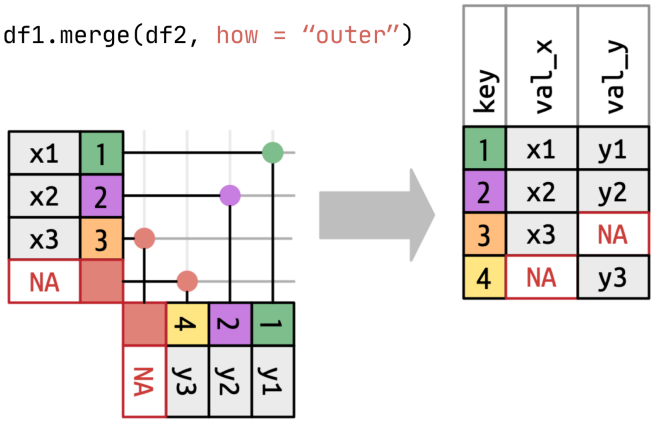


Figure 5. – Illustration du *full outer join*. Source originale: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1]

```
full_merged = emissions.merge(
  filosofi,
  left_on = ["INSEE commune", "dep"],
  right_on = ["CODGEO", "dep"],
  how = "outer"
)
full_merged.head(3)
```

	id_left	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep	id_right	CODGEO	LIBGEO	POPULATION	NIVVIE_MEDIAN	TAUX_PAUVRETE
0	0.0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	795.156501	367.036172	01	0.0	01001	L'Abergement-Clémenciat	860.0	28270.0	NaN
1	1.0	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01	1.0	01002	L'Abergement-de-Varey	270.0	28140.0	NaN
2	2.0	01004	AMBERIEU-EN-BUGEY	499.043526	212.577908	NaN	10313.446515	5314.314445	998.332482	2930.354461	16616.822534	15642.420313	10732.376934	01	2.0	01004	Ambérieu-en-Bugey	15934.0	24210.0	18.0

Comme d'habitude, la traduction en SQL est presque immédiate:

```
SELECT *
FROM emissions
FULL OUTER JOIN filosofi
  ON emissions.`INSEE commune` = filosofi.CODGEO
  AND emissions.dep = filosofi.dep;
```

Cette fois, on a une combinaison de nos trois jeux de données initiaux:

- Le *inner join* ;

- Le *left join* sur les observations sans clé de droite ;
- Le *right join* sur les observations sans clé de gauche ;

```
(
  full_merged['id_left'].isna().sum() + full_merged['id_right'].isna().sum()
) == (
  left_merged['id_right'].isna().sum() + right_merged['id_left'].isna().sum()
)
```

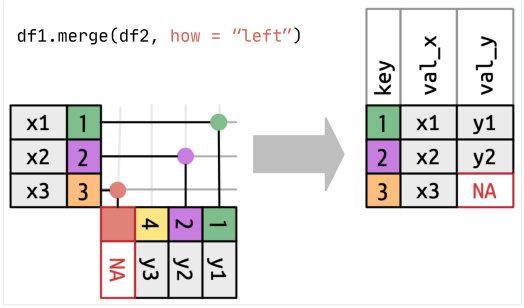
```
np.True_
```

Les colonnes `id_left` et `id_right` n'avaient d'utilité que pour illustrer les jointures ci-dessus. On les retire de nos jeux de données avant de poursuivre, pour ne pas polluer les traitements ultérieurs :

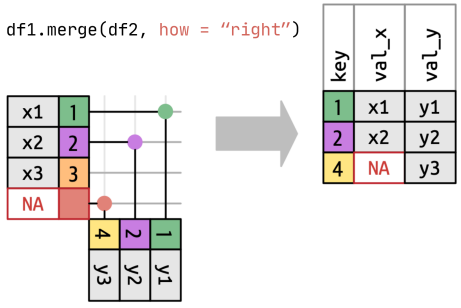
```
emissions = emissions.drop(columns = ['id_left'])
filosofi = filosofi.drop(columns = ['id_right'])
```

4.5 En résumé

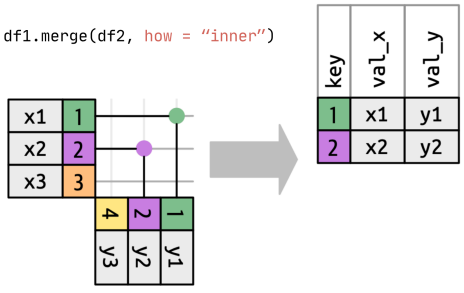
Les quatre principaux types de jointures sont ainsi les suivantes:



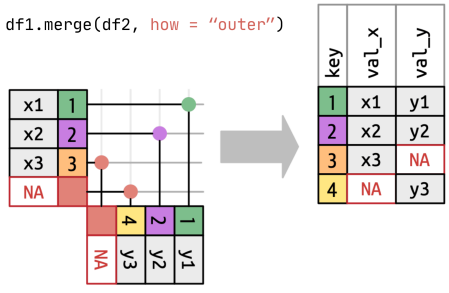
(a) *Left join* (source: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1])



(b) *Right join* (source: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1])



(c) *Inner join* (source: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1])

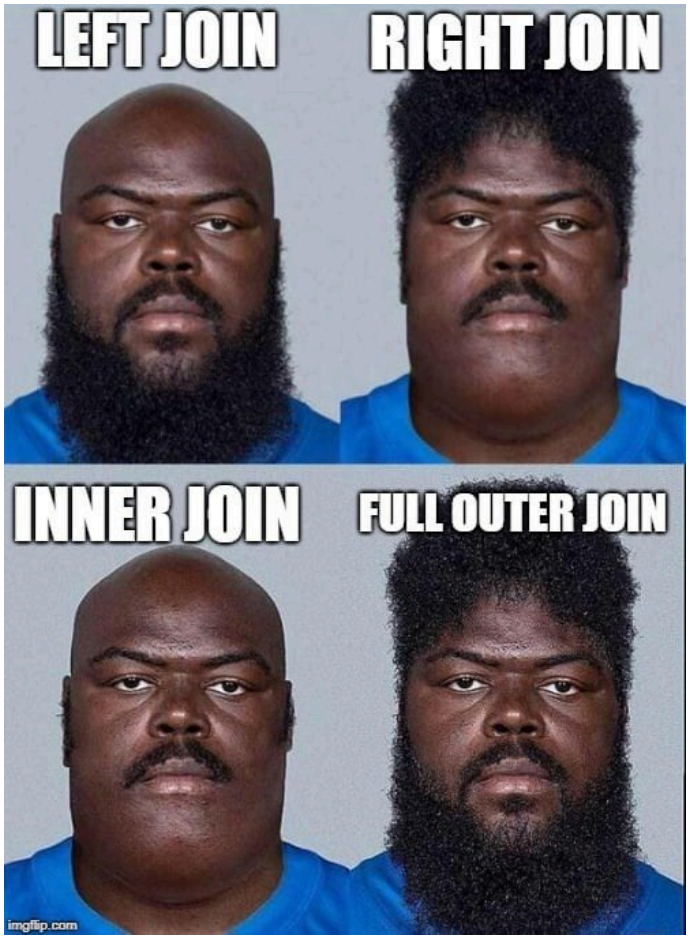


(d) *Full outer join* (source: H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund [1])

Les principales méthodes de fusion de données

Figure 6. –

Dans une version comique, cela donne:



Il existe bien-sûr d'autres types de jointures, basées sur la négation: *semi-join*, *anti-join*, etc. Ces méthodes sont moins fréquemment utilisées mais sont utiles lorsqu'on cherche à déterminer la discordance entre deux jeux de données.

5 Exemples d'identifiants dans les données françaises

Nous avons mis en avant l'intérêt de disposer d'identifiants communs à des bases de données plutôt que se fier à des informations textuelles bruitées. Nous allons maintenant évoquer trois identifiants importants pour les utilisateurs de données françaises:

- Le code officiel géographique, identifiant unique pour des données géographiques
- Le NIR, ou numéro de sécurité sociale, identifiant individuel français
- Le code SIRENE, identifiant unique pour les entreprises ayant une activité en France.

5.1 Le Code officiel géographique (COG): l'identifiant des données géographiques

Pour les données géographiques, il existe de nombreux identifiants selon la problématique d'étude.

Parmi les besoins principaux, on retrouve le fait d'apparier des données géographiques à partir d'un identifiant administratif commun. Par exemple, associer deux jeux de données au niveau communal.

Pour cela, l'identifiant de référence est le code Insee, issu du Code officiel géographique (COG) que nous utilisons depuis le dernier chapitre et que nous aurons amplement l'occasion d'exploiter au cours des différents chapitres de ce cours.

La géographie administrative étant en évolution perpétuelle, la base des code Insee est une base vivante. Le site et les API de l'Insee permettent de récupérer l'historique d'après-guerre afin de pouvoir faire de l'analyse géographique sur longue période.

Les codes postaux ne peuvent être considérés comme un identifiant : ils peuvent regrouper plusieurs communes ou, au contraire, une même commune peut avoir plusieurs codes postaux. Il s'agit d'un système de gestion de la Poste qui n'a pas été construit pour l'analyse statistique.

Pour se convaincre du problème, à partir des données mises à disposition par La Poste, on peut voir que le code postal 11420 correspond à 11 communes:

```
codes_postaux = pd.read_csv(
  "https://datanova.laposte.fr/data-fair/api/v1/datasets/laposte-hexasmal/raw",
  sep = ";", encoding = "latin1",
  dtype = {"Code_postal": "str", "#Code_commune_INSEE": "str"}
)
codes_postaux.loc[codes_postaux['Code_postal'] == "11420"]
```

	#Code_commune_INSEE	Nom_de_la_commune	Code_postal	Libellé_d_acheminement	Ligne_5
3921	11033	BELPECH	11420	BELPECH	NaN
3944	11057	CAHUZAC	11420	CAHUZAC	NaN
4080	11184	LAFAGE	11420	LAFAGE	NaN
4124	11226	MAYREVILLE	11420	MAYREVILLE	NaN
4134	11236	MOLANDIER	11420	MOLANDIER	NaN
4176	11277	PECHARIC ET LE PY	11420	PECHARIC ET LE PY	NaN
4177	11278	PECH LUNA	11420	PECH LUNA	NaN
4182	11283	PEYREFITTE SUR L HERS	11420	PEYREFITTE SUR L HERS	NaN
4189	11290	PLAIGNE	11420	PLAIGNE	NaN
4264	11365	ST SERNIN	11420	ST SERNIN	NaN
4317	11419	VILLAUTOU	11420	VILLAUTOU	NaN

En anticipant sur les compétences développées lors des prochains chapitres, nous pouvons représenter le problème sous forme cartographique en prenant l'exemple de l'Aude. Le code pour produire la carte des codes communes est donné tel quel, il n'est pas développé car il fait appel à des concepts et librairies qui seront présentés lors du prochain chapitre:

```
!pip install cartiflette --quiet
```

```
from cartiflette import carti_download

shp_communes = carti_download(
  values = ["11"],
  crs = 4326,
  borders = "COMMUNE",
  simplification=50,
  filter_by="DEPARTEMENT",
  source="EXPRESS-COG-CARTO-TERRITOIRE",
  year=2022)
```

```
codes_postaux11 = shp_communes.merge(
  codes_postaux,
  left_on = "INSEE_COM",
  right_on = "#Code_commune_INSEE"
)
codes_postaux11 = codes_postaux11.dissolve(by = "Code_postal")
```

Ligne 10

Récupération des contours officiels de l'Aude produits par l'IGN par le biais de la librairie `cartiflette`

Ligne 16

Jointure par le biais du code commune entre les deux sources de données

Ligne 17

On agrège la géométrie au niveau des codes postaux

```
# Carte
ax = shp_communes.plot(color='white', edgecolor='blue', linewidth = 0.5)
ax = codes_postaux11.plot(ax = ax, color='none', edgecolor='black')
ax.set_axis_off()
```

Ligne 1

On crée une carte à partir de nos deux couches

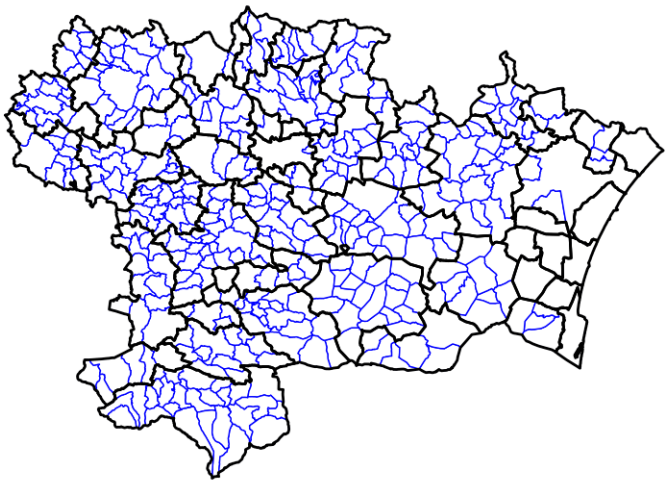


Figure 7. – Géographie des codes postaux et des communes dans l'Aude (11)

5.2 Sirene: l'identifiant dans les données d'entreprises

Pour relier les microdonnées d'entreprises françaises, il existe un numéro unique d'identification : le numéro `Siren`. Il s'agit d'un numéro d'identification dans un répertoire légal d'entreprise indispensable pour toutes démarches juridiques, fiscales...

Pour les entreprises qui possèdent plusieurs établissements - par exemple dans plusieurs villes - il existe un identifiant dérivé qui s'appelle le `Siret`: aux 9 chiffres du numéro Sirene s'ajoutent 5 chiffres d'identifications de l'établissement. D'ailleurs, les administrations publiques sont également concernées par le numéro Siren: étant amenées à effectuer des opérations de marchés (achat de matériel, locations de biens, etc.) elles disposent également d'un identifiant Siren.

Etant inscrits dans des répertoires légaux pour lesquels les citoyens sont publics, les numéros Siren et les noms des entreprises associées sont disponibles en *open data*, par exemple sur annuaire-entreprises.data.gouv.fr/ pour une recherche ponctuelle, sur data.gouv.fr.

Cette base Sirene est une mine d'information, parfois comique, sur les entreprises françaises. Par exemple, le site tif.hair/ s'est amusé à répertorier la part des salons de coiffures proposant des jeux de mots dans le nom du salon.

Lorsqu'un entrepreneur déclare la création d'une entreprise, il reçoit un numéro Sirene et un code d'activité (le code APE) relié à la description qu'il a déclaré de l'activité de son entreprise. Ce code permet de classer l'activité d'une entreprise dans la Nomenclature d'activités françaises (NAF) ce qui servira à l'Insee pour la publication de statistiques sectorielles. En l'occurrence, pour les coiffeurs, le code dans la NAF est `96.02A`.

Il est possible à partir de la base disponible en *open data* d'avoir en quelques lignes de `Python` la liste de tous les coiffeurs puis de s'amuser à explorer ces données (objet du prochain exercice optionnel).

L'exercice suivant, optionnel, propose de s'amuser à reproduire de manière simplifiée le recensement fait par tif.hair/ des jeux de mots dans les salons de coiffure. Il permet de pratiquer quelques méthodes de manipulation textuelle, en avance de phase sur le chapitre consacré aux expressions régulières.

Le jeu de données de l'ensemble des entreprises étant assez volumineux (autour de 4Go en CSV après décompression), il est plus pratique de partir sur un jeu de données au format `Parquet`, plus optimisé (plus de détails sur ce format dans le chapitre d'approfondissement qui lui est consacré).

Pour lire ce type de fichiers de manière optimale, il est conseillé d'utiliser la librairie `DuckDB` qui permet de ne consommer que les données nécessaires et non de télécharger l'ensemble du fichier pour n'en lire qu'une partie comme ce serait le cas avec `Pandas` (voir le dernier chapitre de cette partie, section « Aller au-delà de `Pandas` »). La requête SQL suivante se traduit en langage naturel par l'instruction suivante: *« A partir du fichier `Parquet`, je ne veux que quelques colonnes du fichier pour les coiffeurs (APE: 96.02A) dont le nom de l'entreprise (`denominationUsuelleEtablissement`) est renseigné »*:

```
import duckdb
coiffeurs = duckdb.sql("""
  SELECT
        siren, siret, dateDebut, enseigne1Etablissement, activitePrincipaleEtablissement,
denominationUsuelleEtablissement
  FROM
    read_parquet('https://minio.lab.sspcloud.fr/lgaliana/data/sirene2024.parquet')
  WHERE
    activitePrincipaleEtablissement == '96.02A'
    AND
    denominationUsuelleEtablissement IS NOT NULL
""")
coiffeurs = coiffeurs.df()
```

Ligne 12

Convert the DuckDB dataframe to a Pandas DataFrame.

```
coiffeurs.head(3)
```

	siren	siret	dateDebut	enseigne1Etablissement	activitePrincipaleEtablissement	denominationUsuelleEtablissement
0	024050379	02405037900023	2016-01-14	NaN	96.02A	SOPHA COIFFURE
1	024076481	02407648100019	2017-07-17	NaN	96.02A	SAF COIFFURE
2	047142872	04714287200036	2011-07-31	NaN	96.02A	JENNY

Exercice optionnel : les coiffeurs blagueurs

Dans cet exercice, nous allons considérer exclusivement la variable `denominationUsuelleEtablissement`.

1. Dans cette base, `[ND]` est un code pour valeur manquante. Comme `Python` n'a pas de raison de le savoir *a priori* et donc d'avoir interprété ces valeurs comme étant manquantes, utiliser la méthode `replace` pour remplacer `[ND]` par un champ textuel vide. Recoder également les valeurs manquantes sous forme de champ textuel vide afin d'éviter des erreurs ultérieures liées à l'impossibilité d'appliquer certaines méthodes textuelles aux valeurs manquantes.
2. Rechercher toutes les observations où le terme `tif` apparaît en faisant attention à la capitalisation de la variable. Regarder quelques observations
3. A partir de cet exemple, normaliser les noms des salons en retirant les caractères spéciaux et compter les jeux de mots les plus fréquents

Avec la question 2, on retrouve une liste de jeux de mots assez imaginatifs à partir du terme `tif`:

```
"IMAGINA'TIF, DIMINUT'TIF, A L'INFINI'TIF, TIF'ANNICK, A TRAC TIF, BLEV HAIR TIF, CHRIS TIF,
OBJECTIF PRESTIGE, TIF'ALIZEA HAIRCUT, JS CREA TIF"
```

```
function underlineTif(x) {
  // Use a regular expression to find all occurrences of "tif"
  const modx = x.replace(/TIF/g, match ⇒
     `<span style="text-transform: capitalize; border-bottom: solid 2px blue; margin-bottom:
-2px;">${match}</span>`
  );

  console.log(modx)

  // Return the modified string wrapped in an HTML template
  return html`${modx}`;
}
```

Voici sous une forme plus interactive l'ensemble des coiffeurs qui possèdent les termes `tif` dans le nom de leur entreprise déposée dans les données officielles:

Bien sûr, pour aller plus loin, il faudrait mieux normaliser les données, vérifier que l'information recherchée n'est pas à cheval sur plusieurs colonnes et bien sûr faire de l'inspection visuelle pour détecter les jeux de mots cachés. Mais déjà, en quelques minutes, on a des statistiques partielles sur le phénomène des coiffeurs blagueurs.

5.2.a Le NIR et la question de la confidentialité des identifiants individuels

En ce qui concerne les individus, il existe un identifiant unique permettant de relier ceux-ci dans différentes sources de données : le NIR, aussi connu sous le nom de numéro Insee ou numéro de sécurité sociale. Ce numéro est nécessaire à l'administration pour la gestion des droits à prestations sociales (maladie, vieillesse, famille...). Au-delà de cette fonction qui peut être utile au quotidien, ce numéro est un identifiant individuel unique dans le Répertoire national d'identification des personnes physiques (RNIPP).

Cet identifiant est principalement présent dans des bases de gestion, liées aux fiches de paie, aux prestations sociales, etc. Cependant, *a contrario* du numéro Sirene, celui-ci contient en lui-même plusieurs informations sensibles (Figure 8) - en plus d'être intrinsèquement relié à la problématique sensible des droits à la sécurité sociale.



Figure 8. – Le numéro de sécurité sociale (Source: Améli)

Pour pallier ce problème, a récémment été mis en oeuvre le code statistique non signifiant (CSNS) ou NIR haché, un identifiant individuel anonyme non identifiant. L'objectif de cet identifiant anonymisé est de réduire la dissémination d'une information personnelle qui permettait certes aux fonctionnaires et chercheurs de relier de manière déterministe de nombreuses bases de données mais donnait une information non indispensable aux analystes sur les personnes en question. Y.-L. Bénichou, L. Espinasse, et S. Gilles [2] décrivent plus en détail la manière dont les organisations de la statistique publique française peuvent utiliser ce code unique non signifiant pour associer entre elles diverses sources de données.

6 Exercices d'application

6.1 Pourquoi a-t-on besoin d'un code commune quand on a déjà son nom ?

Cet exercice va revenir un peu en arrière afin de saisir pourquoi nous avons pris comme hypothèse ci-dessus que le code commune était la clé de jointure.

Astuce 1: Vérification des clés de jointure

On commence par vérifier les dimensions des `DataFrames` et la structure de certaines variables clés. En l'occurrence, les variables fondamentales pour lier nos données sont les variables communales. Ici, on a deux variables géographiques: un code commune et un nom de commune.

1. Vérifier les dimensions des *DataFrames*.
2. Identifier dans `filosofi` les noms de communes qui correspondent à plusieurs codes communes et sélectionner leurs codes. En d'autres termes, identifier les `LIBGEO` tels qu'il existe des doublons de `CODGEO` et les stocker dans un vecteur `x` (conseil: faire attention à l'index de `x`).

On se focalise temporairement sur les observations où le libellé comporte plus de deux codes communes différents

- *Question 3.* Regarder dans `filosofi` ces observations.
- *Question 4.* Pour mieux y voir, réordonner la base obtenue par order alphabétique.
- *Question 5.* Déterminer la taille moyenne (variable nombre de personnes: `POPULATION`) et quelques statistiques descriptives de ces données. Comparer aux mêmes statistiques sur les données où libellés et codes communes coïncident.
- *Question 6.* Vérifier les grandes villes (plus de 100 000 personnes), la proportion de villes pour lesquelles un même nom est associé à différents codes commune.
- *Question 7.* Vérifier dans `filosofi` les villes dont le libellé est égal à Montreuil. Vérifier également celles qui contiennent le terme *"Saint-Denis"*.

Ce petit exercice permet donc de se rassurer car les libellés dupliqués sont en fait des noms de commune identiques mais qui ne sont pas dans le même département. Il ne s'agit donc pas d'observations dupliquées. On peut donc se fier aux codes communes, qui eux sont uniques.

6.2 Calculer une empreinte carbone grâce à l'association entre des sources

Astuce 2: Calculer l'empreinte carbone par habitant

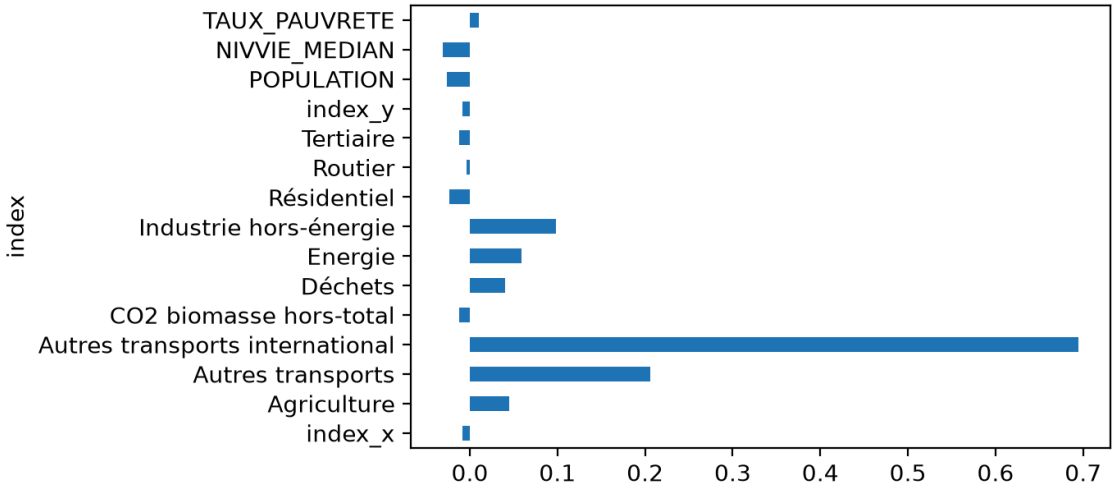
En premier lieu, on va calculer l'empreinte carbone de chaque commune.

1. Créer une variable `emissions` qui correspond aux émissions totales d'une commune
2. Faire une jointure à gauche entre les données d'émissions et les données de cadrage[^note-biais].
3. Calculer l'empreinte carbone (émissions totales / population).

A ce stade nous pourrions avoir envie d'aller vers la modélisation pour essayer d'expliquer les déterminants de l'empreinte carbone à partir de variables communales. Une approche inférentielle nécessite néanmoins pour être pertinente de vérifier en amont des statistiques descriptives.

4. Sortir un histogramme en niveau puis en log de l'empreinte carbone communale.
5. Regarder la corrélation entre les variables de cadrage et l'empreinte carbone. Certaines variables semblent-elles pouvoir potentiellement influer sur l'empreinte carbone ?

A l'issue de la question 5, le graphique des corrélations est le suivant :



Avec une meilleure compréhension de nos données, obtenue grâce à cette association de sources, nous nous rapprochons de la statistique inférentielle. Dans le prochain chapitre, nous allons continuer à exploiter `emissions` et `filosofi`, enrichies l'une par l'autre, pour construire des statistiques descriptives par groupe et restructurer nos données.

Informations additionnelles

ℹ Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
```

```
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
    "kaleido⩾0.2.1",
    "langchain-community⩾0.3.27",
    "loguru==0.7.3",
    "markdown⩾3.8",
    "nbclient⩾0.10.0",
    "nbformat⩾5.10.4",
    "nltk⩾3.9.1",
    "openpyxl⩾3.1.5",
    "pandas⩾3.0",
    "pip⩾25.1.1",
    "plotly⩾6.1.2",
    "plotnine⩾0.15",
    "polars⩾1.8.2",
    "py7zr⩾1.1.3",
    "pyarrow⩾17.0.0",
    "pynsee⩾0.1.8",
    "python-dotenv⩾1.0.1",
    "python-frontmatter⩾1.1.0",
    "pywaffle⩾1.1.1",
    "requests⩾2.32.3",
    "scikit-image⩾0.24.0",
    "scikit-learn⩾1.8.0",
    "scipy⩾1.13.0",
    "seaborn⩾0.13.2",
    "selenium<4.39.0",
    "spacy⩾3.8.4",
    "webdriver-manager⩾4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
```

```
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie

[1] H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund, *R for data science*. " O'Reilly Media, Inc.", 2023.

[2] Y.-L. Bénichou, L. Espinasse, et S. Gilles, « Le Code statistique non signifiant (CSNS) : un service pour faciliter les appariements de fichiers », *Courrier des Statistiques*, nº 9, p. 64-85, juin 2023, [En ligne]. Disponible sur: https://insee.hal.science/hal-05295572